

uPyStore-MicroPythonOS应用商城需求文档和背景介绍

一、项目背景与核心定位

1.1 我们的定位

我们是全球首个聚焦 MCU 场景的 MicroPython 全栈生态服务商，核心产品包含两大主线：

- **uPyPI**：一站式 MicroPython 驱动包仓库，已完成 150+ 款电子模块标准化驱动开发，类比嵌入式领域的 PyPI / npm；
- **uPyOS**：轻量级类安卓嵌入式操作系统，基于 MicroPython + LVGL 构建，运行于 ESP32 系列 MCU，支持图形 UI、应用商城式安装更新，基于 MicroPythonOS。

核心目标：打造智能硬件领域“安卓生态”，降低硬件开发门槛，让零基础用户也能快速完成硬件开发。

1.2 对标项目与核心基准

V1 版本功能对标开源项目 BadgeHub (<https://badge.why2025.org-https://github.com/BadgeHubCrew/badgehub-app>)，该项目为黑客营活动徽章设备提供应用商城服务，我们参考其核心逻辑，结合自身产品特性优化适配，快速实现上线；API 设计部分参考其技术架构，技术栈为 Node.js + PostgreSQL + React。

1.3 核心名词（开发核心关联）

- **MicroPython**：Python 嵌入式精简版本，运行在内存 256KB~8MB 资源受限单片机上，是 uPyOS 运行基础。
- **MCU 单片机**：如 ESP32/ESP32-S3 等无完整系统的芯片，uPyOS 直接运行在此类硬件。
- **uPyOS**：自研嵌入式系统，版本当前 0.9.0，具备桌面启动器、应用安装卸载、系统设置、WiFi 管理等能力。
- **D-Shell**：搭载 uPyOS 的硬件终端设备，类比安卓手机，是 uPyOS 主要运行载体。
- **LVGL**：嵌入式通用图形 UI 引擎，作为 uPyOS 界面渲染底层。
- **.mpk**：uPyOS 应用安装包格式，本质为压缩包，对标安卓 apk，固定内部目录结构不可改动。
- **MANIFEST.JSON**：mpk 包内固定元数据文件，声明应用名称、版本、入口、发布者等，对标安卓清单文件。

- **fullname**: 应用全局唯一标识，反向域名格式，全局不可重复。
- **slug**: fullname 替换小数点为短横线，用于 API 路由、页面地址。
- **revision**: 应用内部版本自增序号，和业务版本号并行，**所有历史版本必须永久保留**，支持设备下载历史版。
- **主动式模块**: 自带蓝牙广播的硬件模块，可主动上报芯片型号与能力，系统自动识别。
- **原始模块**: 普通市面传感器裸模块，无广播能力，依靠扫描 / 手动选择 / 自然语言识别适配，系统设计以这类为基准。
- **device_fingerprint**: 设备匿名唯一指纹，由设备端通过硬件唯一 ID 做 SHA256 哈希生成 64 位十六进制字符串，不可逆，用于去重统计、设备画像、推荐算法底层数据。
- **hardware_tags**: 应用硬件依赖声明 JSON 结构，区分必填 / 可选硬件能力，用于 V2 硬件匹配推荐。
- **capability 硬件能力**: 抽象硬件功能，不绑定具体芯片型号，同一种能力可兼容多款传感器。
- **app_index.json**: 设备端商城唯一静态入口文件，**格式、字段、结构永久不能变更**，否则已出厂设备全部失效。
- **Claude AI Skill**: 底层 AI 代码生成能力，输入硬件画像 + 自然语言需求，输出完整可运行 MicroPython 项目及 mpk 安装包。

三、V1 版本功能需求（快速上线版）

3.1 V1 核心功能清单（必须落地）

序号	功能模块	详细要求说明
1	应用列表	支持分类筛选、关键词模糊搜索、分页加载
2	应用详情	展示图标、应用描述、版本信息、发布者、下载入口
3	版本历史	每一个 revision 独立存档，支持单独查看、下载历史版本
4	开发者提交入驻	第三方账号授权登录 → 上传 mpk 包 → 草稿保存 → 审核 / 正式发布 完整流程
5	图标自动处理	支持任意尺寸图标上传，服务端自动生成两套标准尺寸缩略图
6	应用安装上报	设备安装完成后主动上报， 必须携带 device_fingerprint ，是 V2 所有推荐功能的数据基础
7	下载统计	记录每一次下载行为，汇总展示应用总下载量
8	OS 固件更新分发	提供系统版本查询、固件下载能力，支撑设备端 OTA 升级调用
9	兼容静态接口	固定地址 GET /app_index.json 永久保留、格式永不改动，适配存量设备

3.2 V1 需建表入库、暂不前端展示（为 V2 埋点，必须同步设计）

以下字段 / 数据表 V1 只做数据写入、存储，不开放前台展示，预留 V2 能力：

- `installs_count`：安装次数累加，用于 V2 排行、推荐权重

- `hardware_tags`：开发者提交应用选填字段，V2 硬件匹配核心依据
- `device_profiles`：设备硬件画像完整数据表，V1 完成建表与数据接收
- `stars_count`：应用收藏计数，V1 建表存数据，V2 再开放前端按钮

核心硬性要求：安装上报接口**必须完整携带** `device_fingerprint`，字段缺失或生成规则错误，V2 推荐体系无法复用，需整体重构。

3.3 V1 接口参考规范（按需要可自行更改，仅作参考）

公开接口（无需登录鉴权）：

代码块

```
1 GET /app_index.json
2 GET /api/v1/apps
3 GET /api/v1/apps/{slug}
4 GET /api/v1/apps/{slug}/versions
5 GET /api/v1/apps/{slug}/rev/{n}/files/{path}
6 POST /api/v1/apps/{slug}/rev/{n}/report/install
7 POST /api/v1/devices/profile
8 GET /api/v1/os/latest
9 GET /api/v1/os/releases
```

开发者接口（需身份鉴权）：

代码块

```
1 POST /api/v1/developer/apps
2 PATCH /api/v1/developer/apps/{slug}/draft
3 POST /api/v1/developer/apps/{slug}/draft/icon
4 POST /api/v1/developer/apps/{slug}/draft/files/{path}
5 POST /api/v1/developer/apps/{slug}/publish
6 POST /api/v1/developer/apps/{slug}/new-version
7 DELETE /api/v1/developer/apps/{slug}
8 GET /api/v1/developer/apps/{slug}/stats
```

3.4 V1 核心数据表设计要求（按需要可自行更改，仅作参考）

需按以下实体设计数据表，预留扩展字段，后续尽量不要大改表结构：

- **apps 应用主表**
 - 需包含：slug、应用名、发布者、分类、安装数、下载数、收藏数、hardware_tags、最低 OS 版本、截图、状态、发布时间
- **app_versions 应用版本表**

- 需包含：应用标识、revision 序号、业务版本号、mpk 地址、文件哈希、文件大小、标准图标地址
- **install_reports 安装报表**
 - 需包含：应用标识、revision、device_fingerprint、设备 OS 版本、上报时间；**按应用 + 设备指纹做唯一去重，同一设备只计一次有效安装**
- **device_profiles 设备画像表**
 - 需包含：设备指纹、OS 版本、硬件平台、屏幕分辨率、空闲闪存、检测硬件集合（JSON 结构）、已装应用列表、最后在线时间
- **download_logs 下载日志表**
 - 需包含：应用标识、revision、客户端标识哈希、下载时间
- **users 开发者账号表**
 - 需包含：第三方账号 ID、用户名、邮箱
- **stars 收藏关系表**
 - 需包含：用户 ID、应用标识（V1 建表，V2 开放功能）
- **modules 硬件模块注册表**
 - 需包含：芯片名、展示名、硬件能力集、接口类型、关联 uPyPI 包名、蓝牙广播信息、关键词

四、V2 版本进阶功能需求（研发提前架构预留）

4.1 排行榜与发现

1. 提供多维度榜单：安装最多、下载最多、收藏最多、最新上架、上升最快
2. 各榜单配置独立刷新周期：小时级 / 实时 / 每日刷新
3. 上升最快需按 7 日安装量、应用上线时长做加权计算

4.2 用户收藏能力

1. 登录用户支持收藏 / 取消收藏应用
2. 个人中心展示我的收藏列表，支持分类、搜索
3. 应用详情页展示收藏总数
4. 交互优化：前端乐观更新，后端异步落库

4.3 硬件感知智能推荐

1. 三层降级匹配逻辑必须实现：完整硬件画像→部分设备档案→无设备信息兜底热门榜
2. 基于 `hardware_tags` 与设备检测硬件做能力匹配，屏蔽不兼容应用

3. 设备新增识别硬件后，主动推送适配应用推荐提示
4. 需预留接口：`GET /api/v2/recommend?device={指纹}&limit=10`
`GET /api/v2/recommend?hardware=芯片列表&limit=10`
5. 统一硬件能力枚举标准，后续新增能力统一纳入枚举管理

4.4 协同过滤推荐

1. 基于设备安装行为，实现「安装过此应用的用户还安装了」关联推荐
2. 按日离线计算关联矩阵，结果缓存落地，不实时计算
3. 预留接口：`GET /api/v2/apps/{slug}/related`

4.5 语义搜索

1. 支持自然语言模糊搜索，不仅匹配标题关键词，同时匹配应用描述、硬件能力标签
2. 混合关键词检索 + 向量语义检索，支持中英文内容匹配
3. 应用发布自动生成文本向量入库，预留向量存储与检索能力
4. 预留接口：`GET /api/v2/apps/search?q=xxx&semantic=true`

4.6 AI 代码生成（核心差异化）

1. 四种硬件录入方式任选：绑定设备带入画像、设备 I2C 扫描上报、手动勾选模块、自然语言描述硬件
2. 流程：需求意图拆解→硬件能力匹配→缺失硬件提示→关联 uPyPI 驱动→AI 生成代码→打包 mpk
3. 交付形态：下载安装包、一键推送到设备、在线编辑重打包、保存草稿发布商城
4. 硬件缺失不中断流程，仅给出采购推荐清单
5. 异步任务机制：提交生成请求返回任务 ID，轮询查询结果
6. 按用户身份配置每日调用限流规则
7. 预留接口：`POST /api/v2/generate`
`GET /api/v2/generate/{task_id}`

4.7 自然语言硬件识别

1. 用户文字描述硬件，AI 自动解析匹配候选芯片型号并给出置信度
2. 用户确认后写入设备画像，同步生效推荐逻辑
3. 预留接口：`POST /api/v2/hardware/parse-text`

4.8 开发者数据统计中心

1. 数据维度：下载 / 安装趋势折线图、设备 OS 版本分布、硬件机型分布、应用崩溃错误聚合 Top10

2. 支持 GitHub 仓库 Tag 推送自动触发应用打包、发布流程

五、设备上报强制规范（前后端必须对齐）

5.1 上报事件与接口约定

触发事件	调用接口	上报时机要求
应用安装完成	应用安装上报接口	安装完成立即上报，失败静默重试，不阻塞业务
首次联网 / OS 升级重启	设备画像上报接口	首次联网、系统升级重启后主动上报
检测到新硬件变动	设备画像上报接口	新硬件变化时触发上报

5.2 设备指纹生成规则（固定不可改）

设备端固定采用硬件唯一 ID 做 SHA256 哈希，输出 64 位十六进制字符串，**不可逆、不可解密原始 MAC**，所有上报必须携带此字段。

5.3 上报通用原则

所有上报请求失败仅做静默重试或本地丢弃，**绝对不能弹窗、阻塞用户操作、影响设备主流程**。

六、版本兼容硬性约束（仅作参考）

- 1. V1 所有接口路由、请求参数、响应结构，**永久不做破坏性变更**；
- 2. V2 所有新增功能，全部走 `/api/v2/` 全新路由，不改动老接口；
- 3. 静态文件 `app_index.json` 字段结构、层级、key 名永久锁定，任何版本不得修改；
- 4. 数据表 V1 一次性设计完整，V2 只新增字段、不删除 / 修改原有业务字段。

七、全局开发通用注意点

- 所有版本 revision 永久归档，不允许删除历史安装包、历史版本记录；
- mpk 包内部目录结构、MANIFEST.JSON 规则固定，服务端必须校验格式合法性；
- 开发者上传资源、安装包需做合法性校验、安全检测；
- 所有统计类、推荐类数据，优先采用离线预计算 + 缓存策略，避免高并发下数据库压力；
- 身份授权、开发者权限、接口鉴权体系需统一设计，预留后续权限扩展；
- 架构层面提前预留向量检索、异步任务、定时任务、缓存层能力，适配 V2 后续功能迭代。

(注：内容由 AI 生成，请谨慎参考)